Deep Learning Homework 6

This code is provided for Deep Learning class (601.482/682) Homework 6. For ease of implementation, we recommend working entire in Google Colaboratory.

Modifications made by Hongtao Wu, Suzanna Sia, Hao Ding, Keith Harrigian, and Yiqing Shen.

Problem 1: Segmentation Data Augmentation

```
import os
os.environ['CUDA_LAUNCH_BLOCKING'] = '1'
import numpy as np                # Numpy for array manipulation for c
import torch                      # Pytorch for array manipulation on
import torch.nn as nn
import torch.nn.functional as functional
import torch.utils.data as data
import torchvision
from torch.autograd import Variable
from torch.utils.data import DataLoader
from torchvision import transforms
import cv2
# Image import and display libraries                        # OpenCV fo
import matplotlib.pyplot as plt   # Plotting functions
%matplotlib inline

# Image processing libraries for image feature extractor
from scipy.stats import kurtosis, skew
from scipy.ndimage.filters import generic_filter
from skimage.filters import laplace, gabor
from skimage.filters.rank import entropy
from skimage.morphology import disk
from sklearn.preprocessing import scale

# A few more tools
from sklearn import svm           # SVM classifier library
import os                         # Navigate through directories
import csv                        # Read in a CSV file
import time                       # Timing function
import pickle                     # Saving and loading variables

# Mount Google Drive folder as a local folder
from google.colab import drive
drive.mount('/content/drive')  # Remount
```

```
/tmp/ipython-input-1604491980.py:19: DeprecationWarning: Please import
  from scipy.ndimage.filters import generic_filter
Mounted at /content/drive
```

```
#TODO replace the path with your path in drive
#This usually takes 5 minutes to run
#!cp SegSTRONGC_MLDL.zip ./
!cp -r /content/drive/MyDrive/Colab_Notebooks/SegSTRONGC_MLDL ./
# Uncomment if you feel necesarry
# !rm -r SegSTRONGC_MLDL
# !rm -r __MACOSX/
#!unzip SegSTRONGC_MLDL.zip
```

Hyperparameters

We provide an initial hyper parameter, feel free to change according to your need.

```
#TODO tune your own parameters
batch_size = 10
learning_rate = 0.001
num_epochs = 10
use_gpu = False
if torch.cuda.is_available(): #use gpu if available
  use_gpu = True
  print("using cuda")
```

```
using cuda
```

Data Loaders

We have provided you with some preprocessing code for the images but you should feel free to modify the class however you please to support your training schema.

```
import numpy as np
import os
import os.path as osp
import torch.utils.data as data
import torchvision.transforms as T
from PIL import Image

class SegSTRONGC(data.Dataset):
    def __init__(self, root_folder: str, set_indices: list, subset_in
        '''
            reference dataset loading for SegSTRONGC
```

```
            root_folder: the root_folder of the SegSTRONGC dataset
            set_indices: is the indices for sets to be used
            subset_indices: is the indices for the subsets to be used
            split: 'train', 'val' or 'test'
            domain: the image domains to be loaded.
            image_transforms: any transforms to perform, can add augme
            gt_transforms: list of bool. Indicates whether image_trans
        '''
        self.split = split
        self.root_folder = root_folder
        self.set_indices = set_indices
        self.subset_indices = subset_indices
        self.domains = domains
        self.image_transforms = image_transforms
        self.gt_transforms = gt_transforms

        self.image_paths = []
        self.gt_paths = []

        for set_idx, s in enumerate(self.set_indices):
            for ss in self.subset_indices[set_idx]:
                set_folder = osp.join(self.root_folder, self.split +
                gt_folder = osp.join(set_folder, 'ground_truth')

                for d in self.domains:
                    image_folder = osp.join(set_folder, d)
                    for i in range(300):
                        image_name = str(i) + ".jpg"
                        gt_name = str(i) + ".png"
                        self.image_paths.append(osp.join(image_folder
                        self.gt_paths.append(osp.join(gt_folder, 'lef

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx: int):
        image = np.array(Image.open(self.image_paths[idx])).astype(np
        gt = (np.array(Image.open(self.gt_paths[idx])) / 255).astype(n
        # Apply transformation to image and ground truth
        if self.image_transforms is not None and self.gt_transforms is
            image = self.image_transforms(image)
            gt = self.gt_transforms(gt)
        else:
            image = T.ToTensor()(image)
            gt = T.ToTensor()(gt)
```

```
        return image, gt
```

Model Architecture

Finish building the U-net architecture below.

```
num_classes = 1

def add_conv_stage(dim_in, dim_out, kernel_size=3, stride=1, padding=1
  if useBN:
    return nn.Sequential(
      nn.Conv2d(dim_in, dim_out, kernel_size=kernel_size, stride=stri
      nn.BatchNorm2d(dim_out),
      nn.LeakyReLU(0.1),
      nn.Conv2d(dim_out, dim_out, kernel_size=kernel_size, stride=str
      nn.BatchNorm2d(dim_out),
      nn.LeakyReLU(0.1)
    )
  else:
    return nn.Sequential(
      nn.Conv2d(dim_in, dim_out, kernel_size=kernel_size, stride=stri
      nn.ReLU(),
      nn.Conv2d(dim_out, dim_out, kernel_size=kernel_size, stride=str
      nn.ReLU()
    )

def add_merge_stage(ch_coarse, ch_fine, in_coarse, in_fine, upsample)
  conv = nn.ConvTranspose2d(ch_coarse, ch_fine, 4, 2, 1, bias=False)
  torch.cat(conv, in_fine)

  return nn.Sequential(
    nn.ConvTranspose2d(ch_coarse, ch_fine, 4, 2, 1, bias=False)
  )
  upsample(in_coarse)

def upsample(ch_coarse, ch_fine):
  return nn.Sequential(
    nn.ConvTranspose2d(ch_coarse, ch_fine, 4, 2, 1, bias=False),
    nn.ReLU()
  )
```

```
class unet(nn.Module):
  def __init__(self, useBN=False):
    super(unet, self).__init__()
    # Downgrade stages
    self.conv1   = add_conv_stage(3, 32, useBN=useBN)
    self.conv2   = add_conv_stage(32, 64, useBN=useBN)
    self.conv3   = add_conv_stage(64, 128, useBN=useBN)
    self.conv4   = add_conv_stage(128, 256, useBN=useBN)
    self.conv5   = add_conv_stage(256, 512, useBN=useBN)
    # Upgrade stages
    self.conv4m = add_conv_stage(512, 256, useBN=useBN)
    self.conv3m = add_conv_stage(256, 128, useBN=useBN)
    self.conv2m = add_conv_stage(128,  64, useBN=useBN)
    self.conv1m = add_conv_stage( 64,  32, useBN=useBN)
    # Maxpool
    self.max_pool = nn.MaxPool2d(2)
    # Upsample layers
    self.upsample54 = upsample(512, 256)
    self.upsample43 = upsample(256, 128)
    self.upsample32 = upsample(128,  64)
    self.upsample21 = upsample(64 ,  32)

    ## TODO Design your last layer & activations
    self.final_conv = nn.Conv2d(32, num_classes, kernel_size=1)

    # Choose activation based on task
    if num_classes == 1:  # Binary segmentation
        self.final_activation = nn.Sigmoid()
    else:  # Multi-class segmentation
        self.final_activation = nn.Softmax(dim=1)


  def forward(self, x):
    #TODO implement forward function
    # Encoder
    x1 = self.conv1(x)              # (N, 32, H, W)
    p1 = self.max_pool(x1)          # (N, 32, H/2, W/2)

    x2 = self.conv2(p1)             # (N, 64, H/2, W/2)
    p2 = self.max_pool(x2)          # (N, 64, H/4, W/4)

    x3 = self.conv3(p2)             # (N, 128, H/4, W/4)
    p3 = self.max_pool(x3)          # (N, 128, H/8, W/8)

    x4 = self.conv4(p3)             # (N, 256, H/8, W/8)
```

```
        p4 = self.max_pool(x4)          # (N, 256, H/16, W/16)

        x5 = self.conv5(p4)             # (N, 512, H/16, W/16)

        # Decoder with skip connections
        u4 = self.upsample54(x5)        # (N, 256, H/8, W/8)
        m4 = torch.cat([u4, x4], dim=1) # (N, 512, H/8, W/8)
        x4m = self.conv4m(m4)           # (N, 256, H/8, W/8)

        u3 = self.upsample43(x4m)       # (N, 128, H/4, W/4)
        m3 = torch.cat([u3, x3], dim=1) # (N, 256, H/4, W/4)
        x3m = self.conv3m(m3)           # (N, 128, H/4, W/4)

        u2 = self.upsample32(x3m)       # (N, 64, H/2, W/2)
        m2 = torch.cat([u2, x2], dim=1) # (N, 128, H/2, W/2)
        x2m = self.conv2m(m2)           # (N, 64, H/2, W/2)

        u1 = self.upsample21(x2m)       # (N, 32, H, W)
        m1 = torch.cat([u1, x1], dim=1) # (N, 64, H, W)
        x1m = self.conv1m(m1)           # (N, 32, H, W)

        logits = self.final_conv(x1m)   # (N, num_classes, H, W)
        return self.final_activation(logits)
```

Here defines trainning functions, diceloss functions and dice evaluation functions

```
    def trainning(model, trainning_dataloader, validation_dataloader, num_
        if use_gpu:
          model.cuda()
        lr_changed = False
        trainning_losses = []
        validation_losses = []
        total_training_loss = 0
        total_val_loss = 0
        total_training_iteration = 0
        total_val_iteration = 0
        for epoch in range(num_epochs):
            i = 0
            model.train()
            for data in trainning_dataloader:
              img,y = data
              if use_gpu:
```

```
            img = img.cuda()
            y = y.cuda()
          out = model(img)
          model.zero_grad()
          loss = criterion(out, y)
          total_training_loss += loss.item()
          loss.backward()
          optimizer.step()
          i = i+1
          total_training_iteration += 1
          if total_training_iteration % 100 == 99:
            trainning_losses.append(total_training_loss / total_train
        if epoch % 5 == 4:
            print("learning_rate decayed")
            for param_group in optimizer.param_groups:
              param_group['lr'] *= 0.1
        model.eval()
        for data in validation_dataloader:
          img,y = data
          if use_gpu:
            img = img.cuda()
            y = y.cuda()
          out = model(img)
          model.zero_grad()
          loss = criterion(out, y)
          total_val_loss += loss.item()
          total_val_iteration += 1
          if total_val_iteration % 100 == 99:
            validation_losses.append(total_val_loss / total_val_itera
        print("epoch:",epoch,"training_loss:",total_training_loss / t
        torch.save(model.state_dict(), filename)
    plt.plot(trainning_losses)
    plt.show()
    plt.plot(validation_losses)
    plt.show()
```

DICE Score and DICE Loss

Finish implementing the DICE score function below and then write a Dice Loss function that you can use to update your model weights.

```
def DICE(model, test_dataloader, smooth=1e-10):
    dice = []
    model.eval()
    for data in test_dataloader:
        img, target = data
        if use_gpu:
            img = img.cuda()
            target = target.cuda()
        predict = model(img) > 0.5
        num = 2 * (predict * target).sum()
        denum = predict.sum() + target.sum()
        dice.append(((num + smooth) / (denum + smooth)).item())
    m_dice = np.mean(dice)
    return m_dice

def DICELoss(scores, target):
    # TODO complete dice loss to calculate dice of the segmented tool
    # Ensure same dtype/shape
    target = target.float()
    scores = scores.float()

    # Flatten per-sample
    scores_flat = scores.contiguous().view(scores.size(0), -1)
    target_flat = target.contiguous().view(target.size(0), -1)

    intersection = (scores_flat * target_flat).sum(dim=1)
    denom = scores_flat.sum(dim=1) + target_flat.sum(dim=1)

    dice = (2.0 * intersection) / denom
    loss = 1.0 - dice
    return loss.mean()
```

define transforms and vanilla dataset for trainning

```
root_folder = "./SegSTRONGC_MLDL"
size = (272, 480)
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]
train_set_indices = [3, 4, 5, 7, 8]
train_subset_indices = [[0, 2], [0, 1, 2], [0, 2], [0, 1], [1, 2]]

train_image_transforms = transforms.Compose([
    transforms.ToTensor(),
```

```
        transforms.Resize(size, interpolation=transforms.InterpolationMode.
        transforms.Normalize(mean=mean, std=std)
    ])
    train_gt_transforms = transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize(size, interpolation=transforms.InterpolationMode.
    ])
    segmentation_trainning_dataset = SegSTRONGC(
        root_folder = root_folder,
        set_indices = train_set_indices,
        subset_indices = train_subset_indices,
        split = 'train',
        domains = ['regular'],
        image_transforms = train_image_transforms,
        gt_transforms = train_gt_transforms)
    segmentation_trainning_dataloader = DataLoader(segmentation_trainning_d

    val_set_indices = [1]
    val_subset_indices = [[0]]

    val_image_transforms = transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize(size, interpolation=transforms.InterpolationMode.
        transforms.Normalize(mean=mean, std=std)
    ])
    val_gt_transforms = transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize(size, interpolation=transforms.InterpolationMode.
    ])
    segmentation_validation_dataset = SegSTRONGC(
        root_folder = root_folder,
        set_indices = val_set_indices,
        subset_indices = val_subset_indices,
        split = 'val',
        domains = ['regular'],
        image_transforms = val_image_transforms,
        gt_transforms = val_gt_transforms)

    segmentation_validation_dataloader = DataLoader(segmentation_validation

    test_set_indices = [9]
    test_subset_indices = [[0]]

    test_image_transforms = transforms.Compose([
        transforms.ToTensor(),
```

```
        transforms.Resize(size, interpolation=transforms.InterpolationMode.
        transforms.Normalize(mean=mean, std=std)
    ])
    test_gt_transforms = transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize(size, interpolation=transforms.InterpolationMode.
    ])
    segmentation_test_dataset = SegSTRONGC(
        root_folder = root_folder,
        set_indices = test_set_indices,
        subset_indices = test_subset_indices,
        split = 'test',
        domains = ['regular'],
        image_transforms = test_image_transforms,
        gt_transforms = test_gt_transforms)

    segmentation_test_dataloader = DataLoader(segmentation_test_dataset, ba

    segmentation_test_dataset_blood = SegSTRONGC(
        root_folder = root_folder,
        set_indices = test_set_indices,
        subset_indices = test_subset_indices,
        split = 'test',
        domains = ['blood'],
        image_transforms = test_image_transforms,
        gt_transforms = test_gt_transforms)

    segmentation_test_dataloader_blood = DataLoader(segmentation_test_datas
```

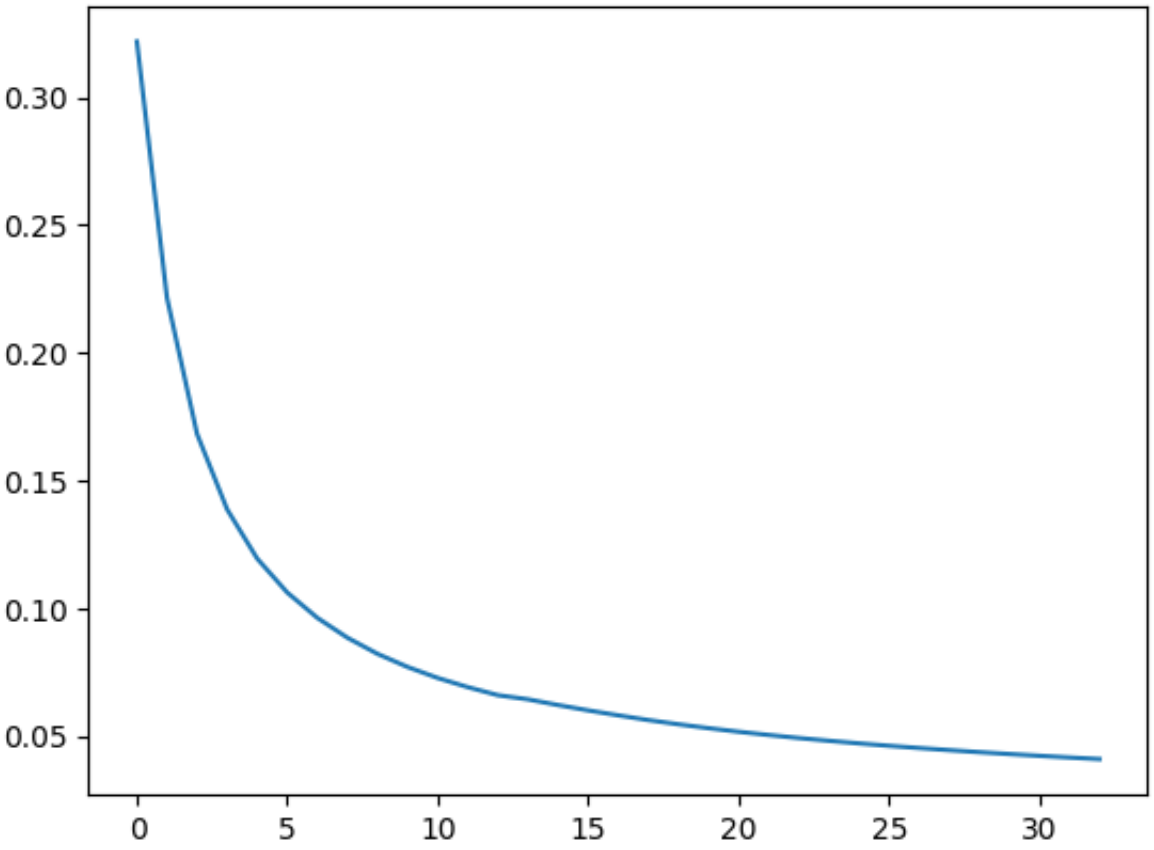
```
def show_demo(model, test_dataset_loader, num=10, std=std, mean=mean)
    model.eval()
    count = 0
    for demo in test_dataset_loader:
        demo_input, demo_target = demo
        if use_gpu:
           demo_input = demo_input.cuda()
        demo_output = model(demo_input)
        #denomalize the input image
        for i in range(demo_input.shape[0]):
            demo_image = demo_input[i].permute(1,2,0).detach().cpu().r
            demo_image[:,:,0] = demo_image[:,:,0]*std[0]+mean[0]
            demo_image[:,:,1] = demo_image[:,:,1]*std[1]+mean[1]
            demo_image[:,:,2] = demo_image[:,:,2]*std[2]+mean[2]
            plt.subplot(1, 3, 1)
            plt.imshow(demo_image)
            plt.axis("off")
            plt.subplot(1, 3, 2)
            plt.imshow(demo_output[i].detach().cpu().numpy().squeeze(
            plt.axis("off")
            plt.subplot(1, 3, 3)
            plt.imshow(demo_target[i].detach().numpy().squeeze())
            plt.axis("off")
            plt.show()
        if count >= num:
          break
        count += 1
```

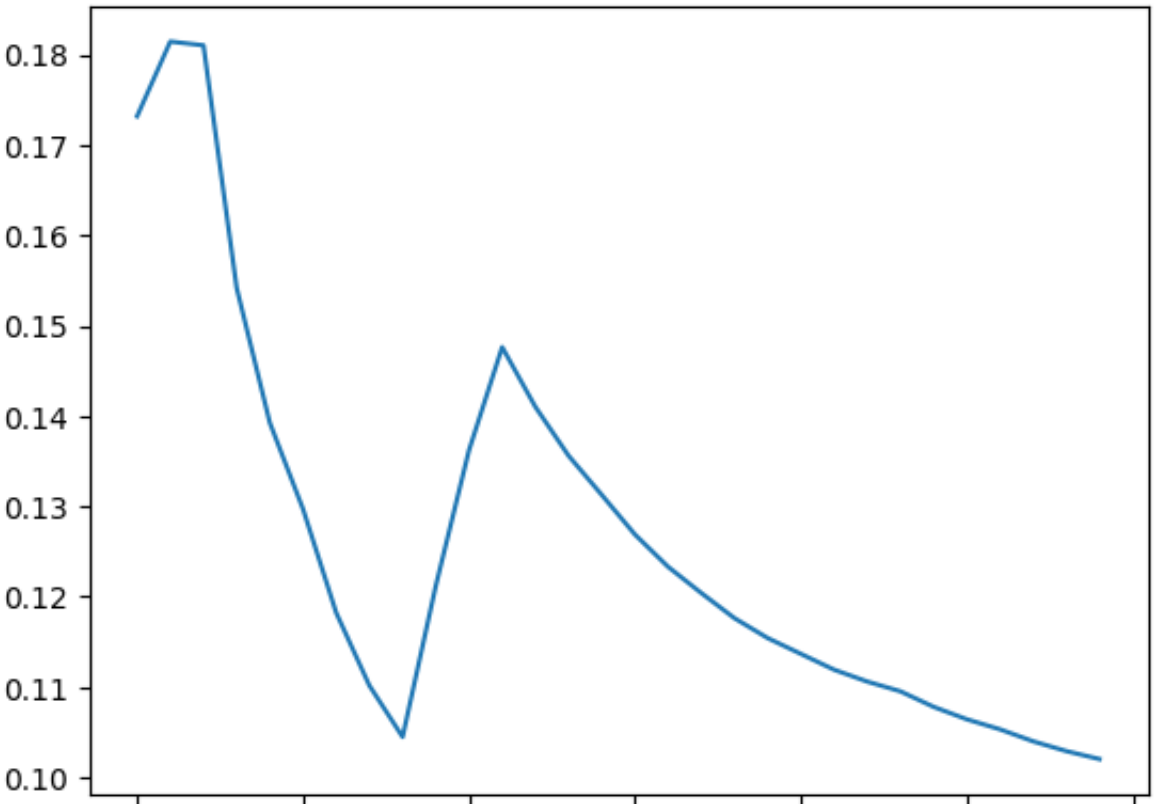
Training vanilla model on vanilla dataset

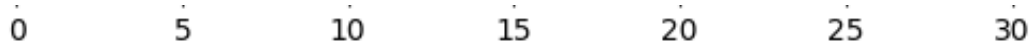
```
segmentation_model = unet(useBN=True)
dice_criterion = DICELoss
segmentation_optimizer = torch.optim.Adam(segmentation_model.paramete
trainning(segmentation_model, segmentation_trainning_dataloader, segm
```

```
epoch: 0 training_loss: 0.1576888627175129 validation_loss: 0.1809191
epoch: 1 training_loss: 0.09984733478984598 validation_loss: 0.129596
epoch: 2 training_loss: 0.07736912154489094 validation_loss: 0.104414
epoch: 3 training_loss: 0.06579286282261212 validation_loss: 0.147730
learning_rate decayed
epoch: 4 training_loss: 0.05897769495619066 validation_loss: 0.131222
epoch: 5 training_loss: 0.05325387627864727 validation_loss: 0.120373
epoch: 6 training_loss: 0.04899977050253968 validation_loss: 0.113649
```

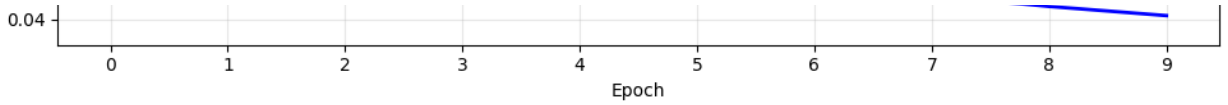
```
epoch: 7 training_loss: 0.045714111927416964 validation_loss: 0.10950
epoch: 8 training_loss: 0.04306122215232665 validation_loss: 0.105306
learning_rate decayed
epoch: 9 training_loss: 0.040863638757022494 validation_loss: 0.10201
```





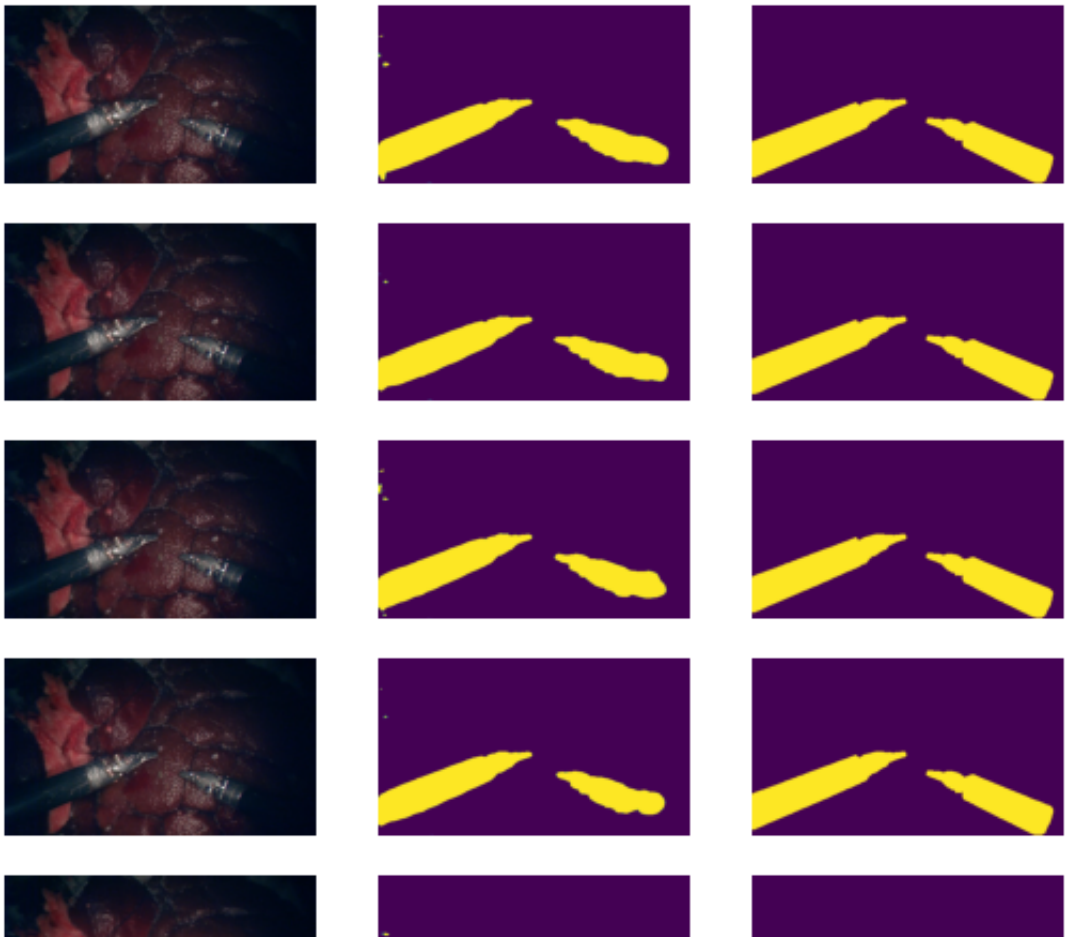


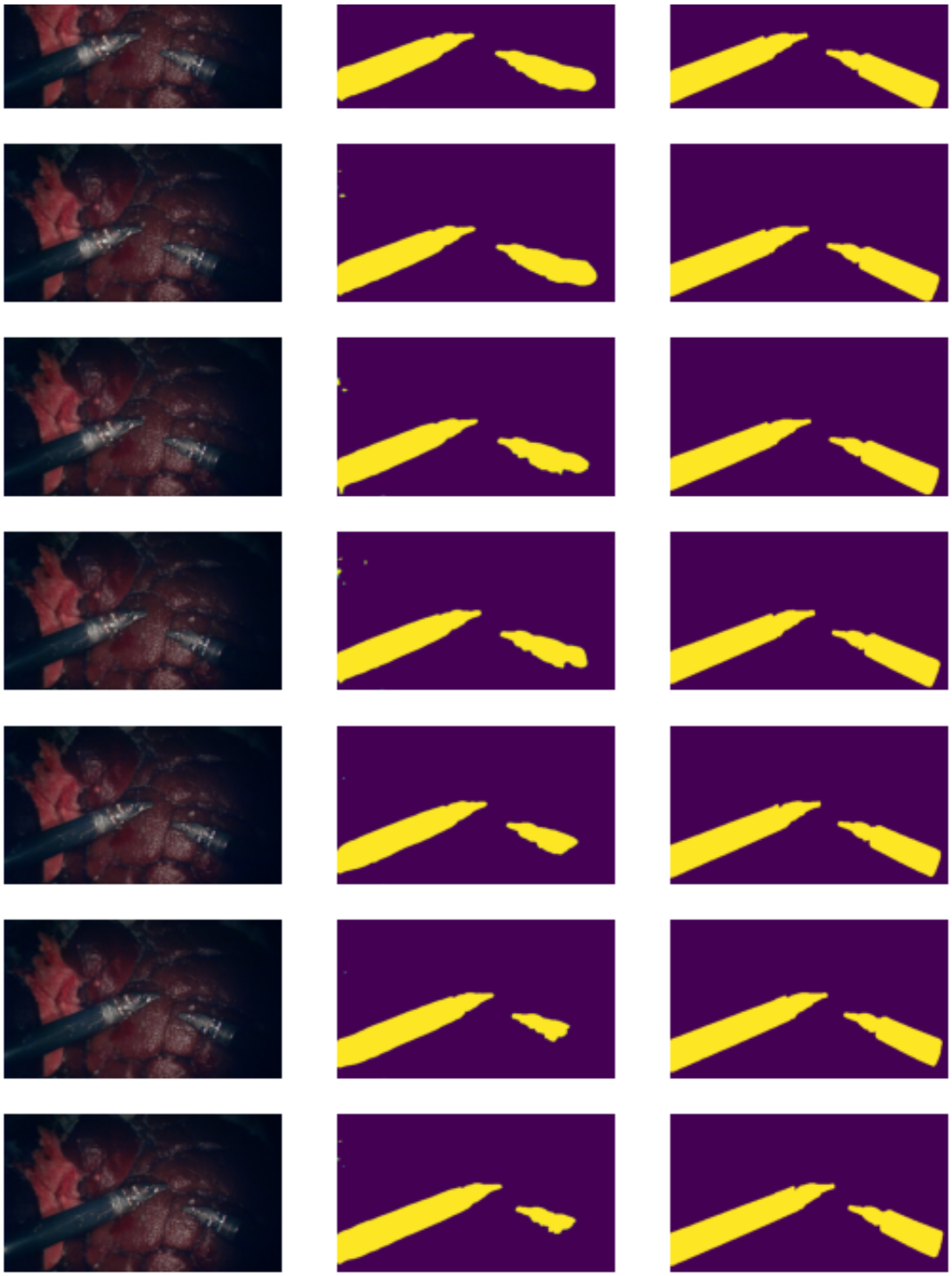

```
import matplotlib.pyplot as plt

# Loss data
epochs = list(range(10))
training_loss = [0.1576888627175129, 0.09984733478984598, 0.07736912154
                 0.05897769495619066, 0.05325387627864727, 0.0489997705
                 0.04306122215232665, 0.040863638757022494]
validation_loss = [0.18091911554336548, 0.12959601769844692, 0.10441453
                   0.13122253354390462, 0.12037360234393014, 0.11364935
                   0.10530610980810942, 0.10201172381639481]

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(epochs, training_loss, 'b-', label='Training Loss', linewidth=
plt.plot(epochs, validation_loss, 'r-', label='Validation Loss', linewi

plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(epochs)

plt.tight_layout()
plt.show()
```




```
segmentation_model.load_state_dict(torch.load("final_vanilla_model.pt
print(DICE(segmentation_model, segmentation_test_dataloader))
show_demo(segmentation_model, segmentation_test_dataloader)
```

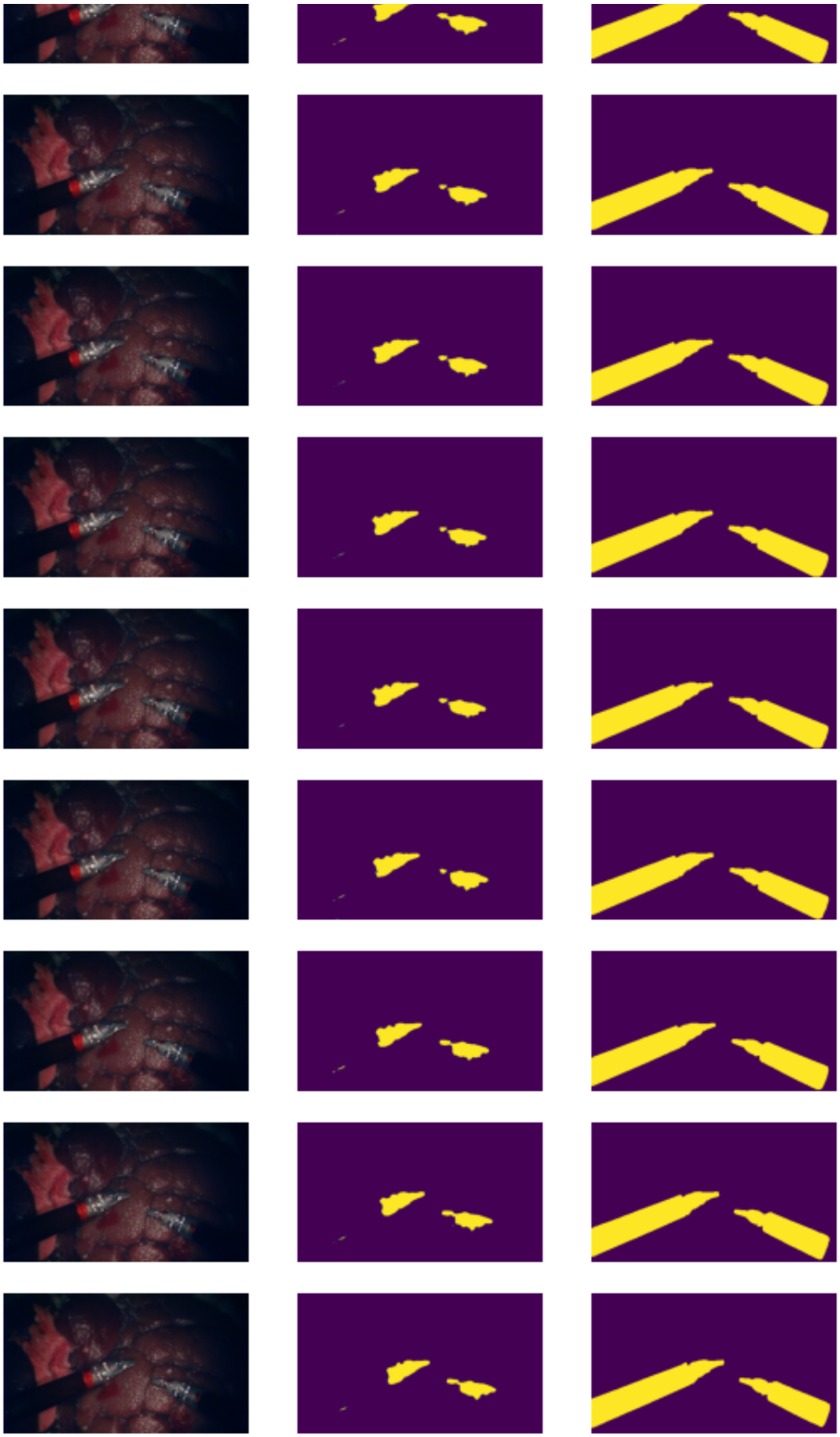
```
0.9429992165168126
```





```
segmentation_model.load_state_dict(torch.load("final_vanilla_model.ptl
print(DICE(segmentation_model, segmentation_test_dataloader_blood))
show_demo(segmentation_model, segmentation_test_dataloader_blood)
```

```
0.46090797225634256
```



Do data augmentation for better trainning

Think about what data augmentations you would like to use to help with blood situation.

```
# #TODO add transformations in training dataset,
# # Carefully think about whether the it is suitable for segmentation
import random
from torchvision.transforms import functional as TF

# Shared transform for geometric operations (applied to both image and
class SharedRandomTransform:
    def __init__(self, flip_p=0.5):
        self.flip_p = flip_p
        self.current_params = None

    def apply_to_image(self, img_tensor):
        # Store params for this sample
        self.current_params = {
            'do_flip': random.random() < self.flip_p,
            'brightness': random.uniform(0.8, 1.2),
            'contrast': random.uniform(0.8, 1.2)
        }

        img = img_tensor.clone()

        # Apply flip (shared with mask)
        if self.current_params['do_flip']:
            img = TF.hflip(img)
```

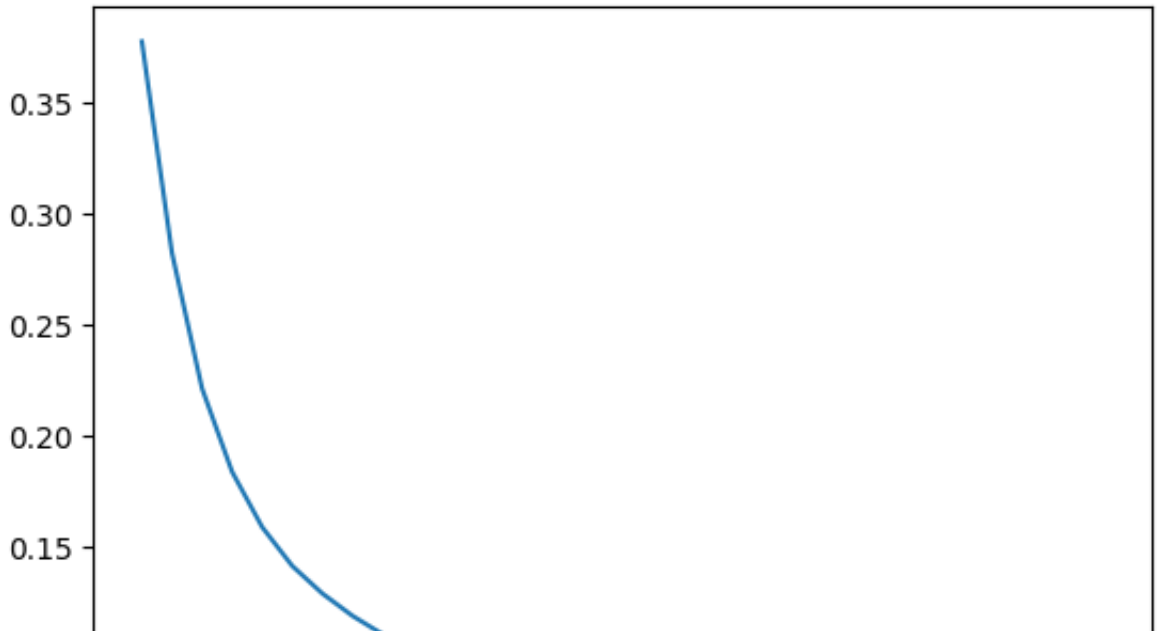
```
            # Apply photometric changes (image only)
            img = TF.adjust_brightness(img, self.current_params['brightnes
            img = TF.adjust_contrast(img, self.current_params['contrast']

            return img

        def apply_to_mask(self, mask_tensor):
            if self.current_params is None:
                return mask_tensor

            mask = mask_tensor.clone()

            # Apply only the geometric transform (flip)
            if self.current_params['do_flip']:
                mask = TF.hflip(mask)

            # Reset for next sample
            self.current_params = None
            return mask

    # Create shared transform instance
    shared_transform = SharedRandomTransform(flip_p=0.5)

    # IMAGE transforms with minimal augmentation
    train_image_transforms_augmented = transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize(size, interpolation=transforms.InterpolationMod
        lambda x: shared_transform.apply_to_image(x),  # Apply shared tra
        transforms.ColorJitter(brightness=0.2, contrast=0.2),  # Small ad
        transforms.Normalize(mean=mean, std=std),
    ])

    # MASK transforms (only geometric operations shared with image)
    train_gt_transforms_augmented = transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize(size, interpolation=transforms.InterpolationMod
        lambda x: shared_transform.apply_to_mask(x),  # Apply shared geom
    ])

    segmentation_trainning_dataset_augmented = SegSTRONGC(
        root_folder = root_folder,
        set_indices = train_set_indices,
        subset_indices = train_subset_indices,
        split = 'train',
        domains = ['regular'],
```

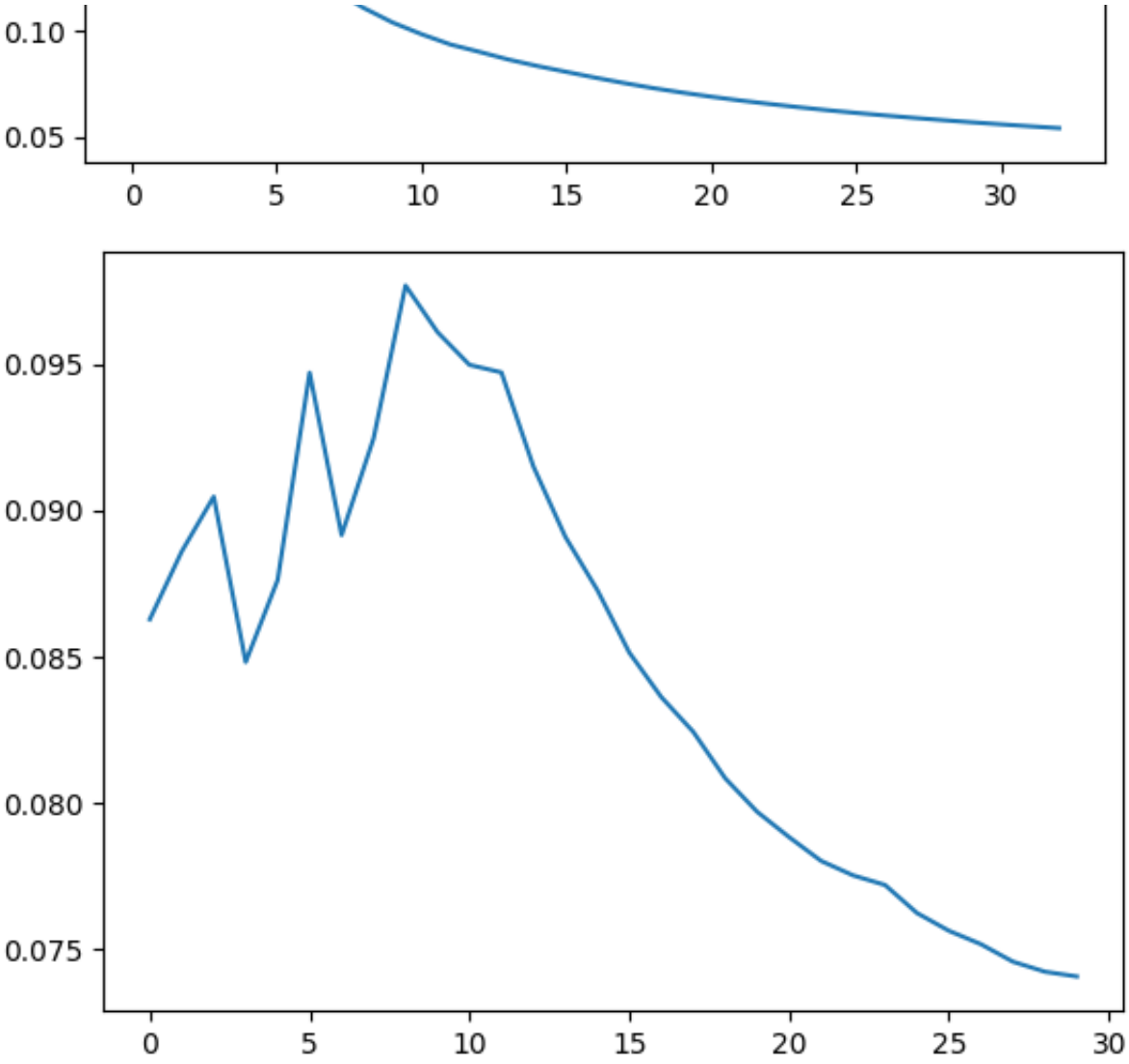
```
    image_transforms = train_image_transforms_augmented,
    gt_transforms = train_gt_transforms_augmented
)

segmentation_trainning_dataloader_augmented = DataLoader(
    segmentation_trainning_dataset_augmented,
    batch_size=batch_size,
    shuffle=True
)
```

Retrain with augmented dataset

```
segmentation_model_augmented = unet(useBN=True)
dice_criterion = DICELoss
segmentation_optimizer_augmented = torch.optim.Adam(segmentation_mode
trainning(segmentation_model_augmented, segmentation_trainning_datalo
```

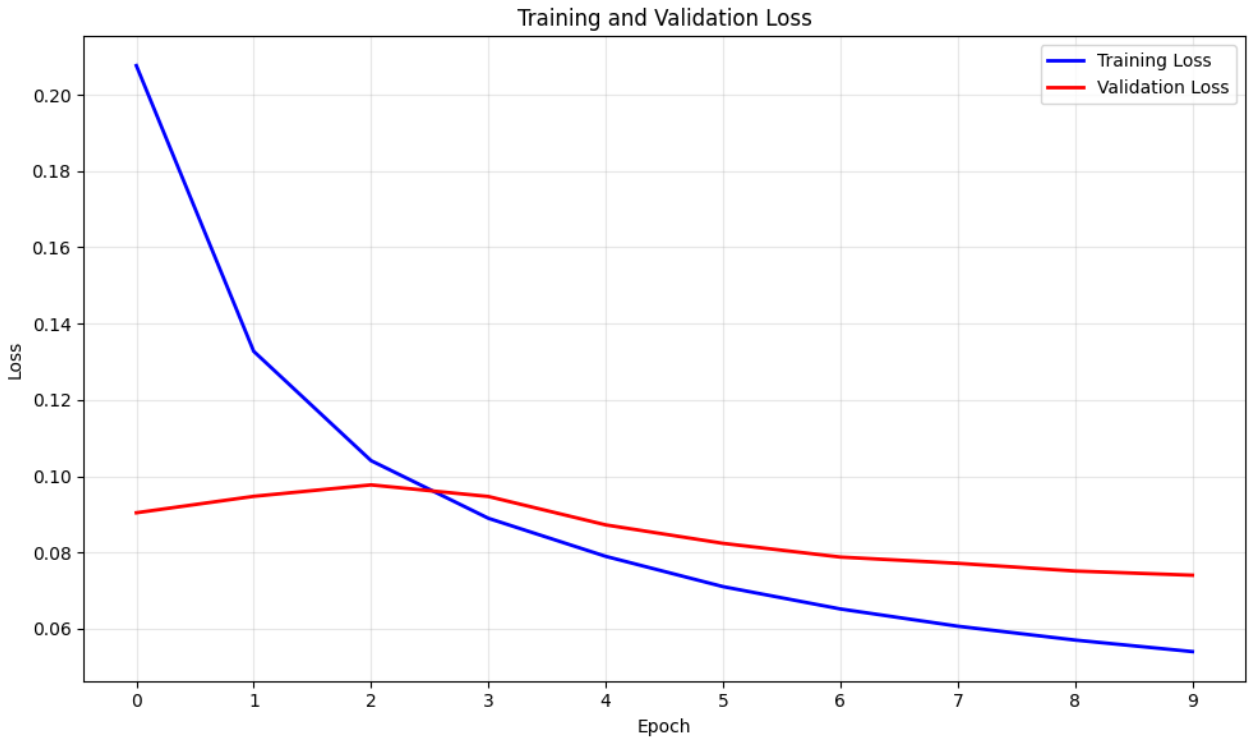
```
epoch: 0 training_loss: 0.2076556461552779 validation_loss: 0.0904189
epoch: 1 training_loss: 0.132756099353234 validation_loss: 0.09472977
epoch: 2 training_loss: 0.10409158815563929 validation_loss: 0.097726
epoch: 3 training_loss: 0.08897377999471218 validation_loss: 0.094687
learning_rate decayed
epoch: 4 training_loss: 0.0789935174945629 validation_loss: 0.0872429
epoch: 5 training_loss: 0.07105480546244618 validation_loss: 0.082403
epoch: 6 training_loss: 0.06519765762120357 validation_loss: 0.078804
epoch: 7 training_loss: 0.06067915343623044 validation_loss: 0.077173
epoch: 8 training_loss: 0.05704814558880096 validation_loss: 0.075147
learning_rate decayed
epoch: 9 training_loss: 0.05401801217911822 validation_loss: 0.074047
```






```
import matplotlib.pyplot as plt

# Loss data
epochs = list(range(10))
training_loss = [0.2076556461552779, 0.132756099353234, 0.10409158815
                 0.0789935174945629, 0.07105480546244618, 0.065197657
                 0.05704814558880096, 0.05401801217911822]
validation_loss = [0.09041899859905243, 0.09472977538903554, 0.097726
                   0.08724294316768647, 0.0824035816391309, 0.0788048
                   0.07514771132557481, 0.07404731078942617]

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(epochs, training_loss, 'b-', label='Training Loss', linewidth
plt.plot(epochs, validation_loss, 'r-', label='Validation Loss', linew
```

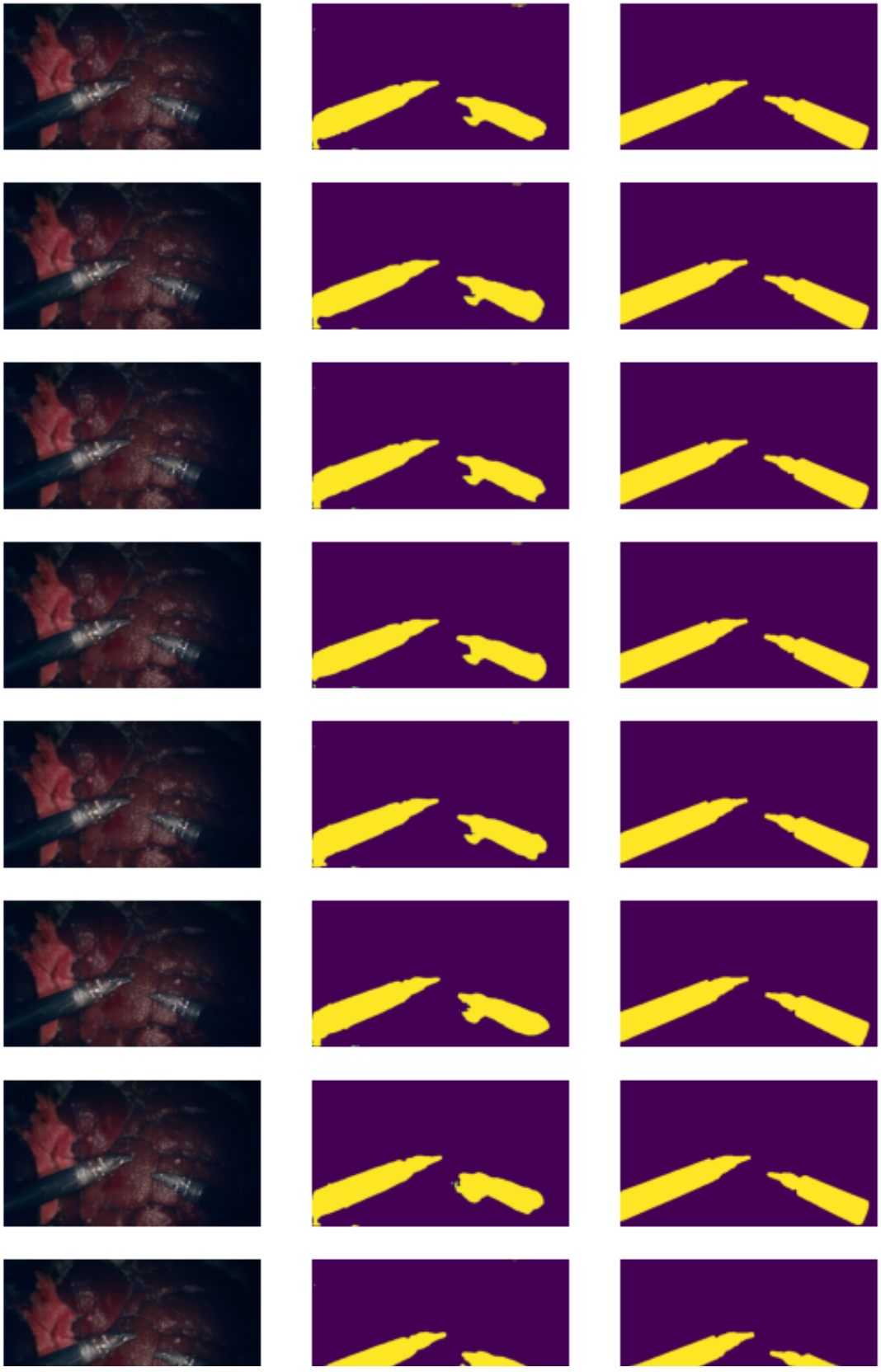
```
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(epochs)

plt.tight_layout()
plt.show()
```

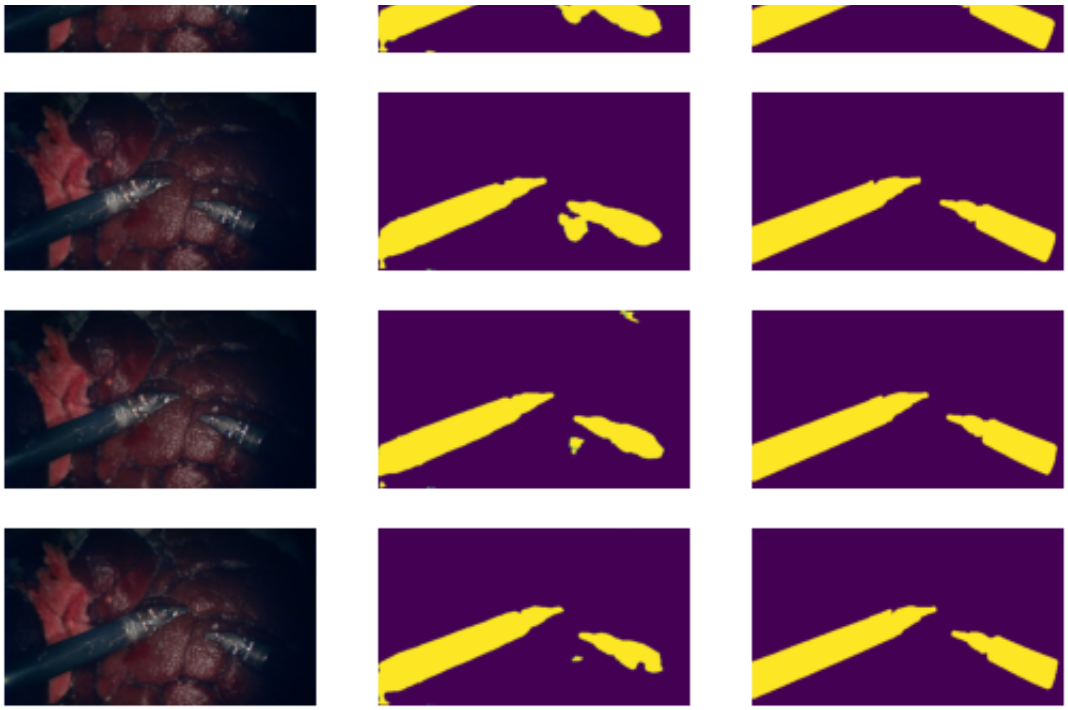



```
segmentation_model_augmented.load_state_dict(torch.load("final_augmen
```

```
print(DICE(segmentation_model_augmented, segmentation_test_dataloader
show_demo(segmentation_model_augmented, segmentation_test_dataloader)
```

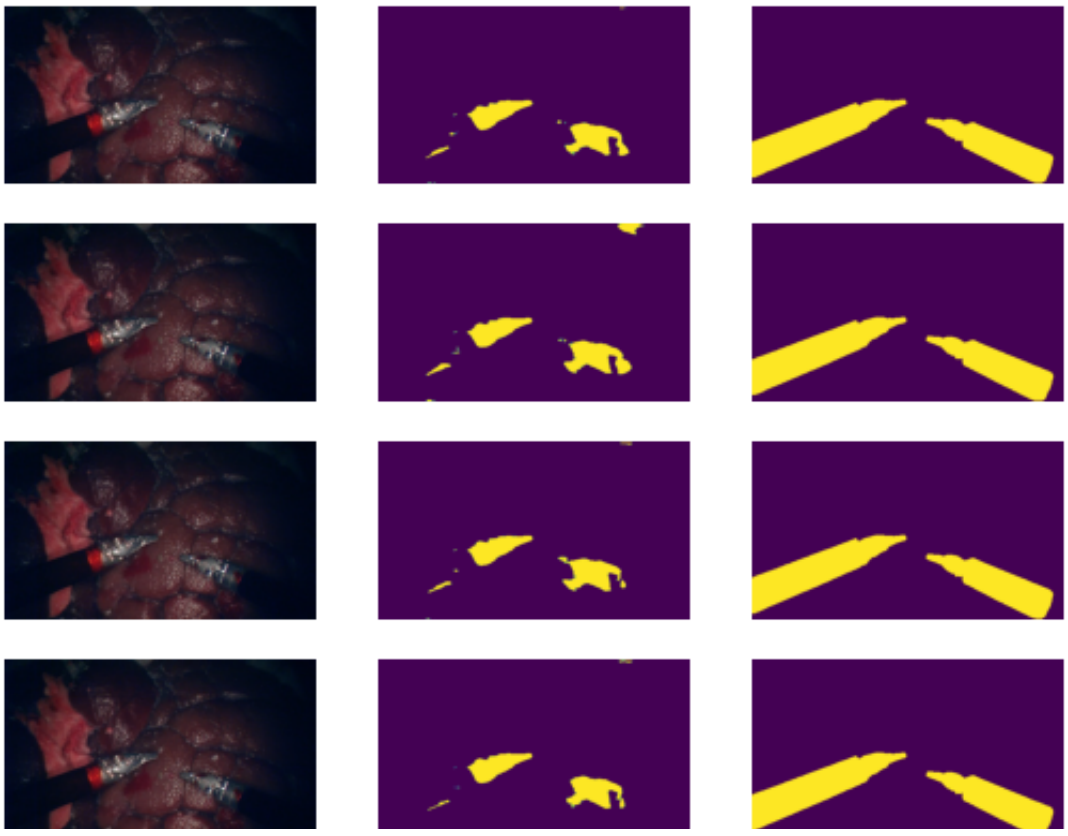
```
0.945537406206131
```

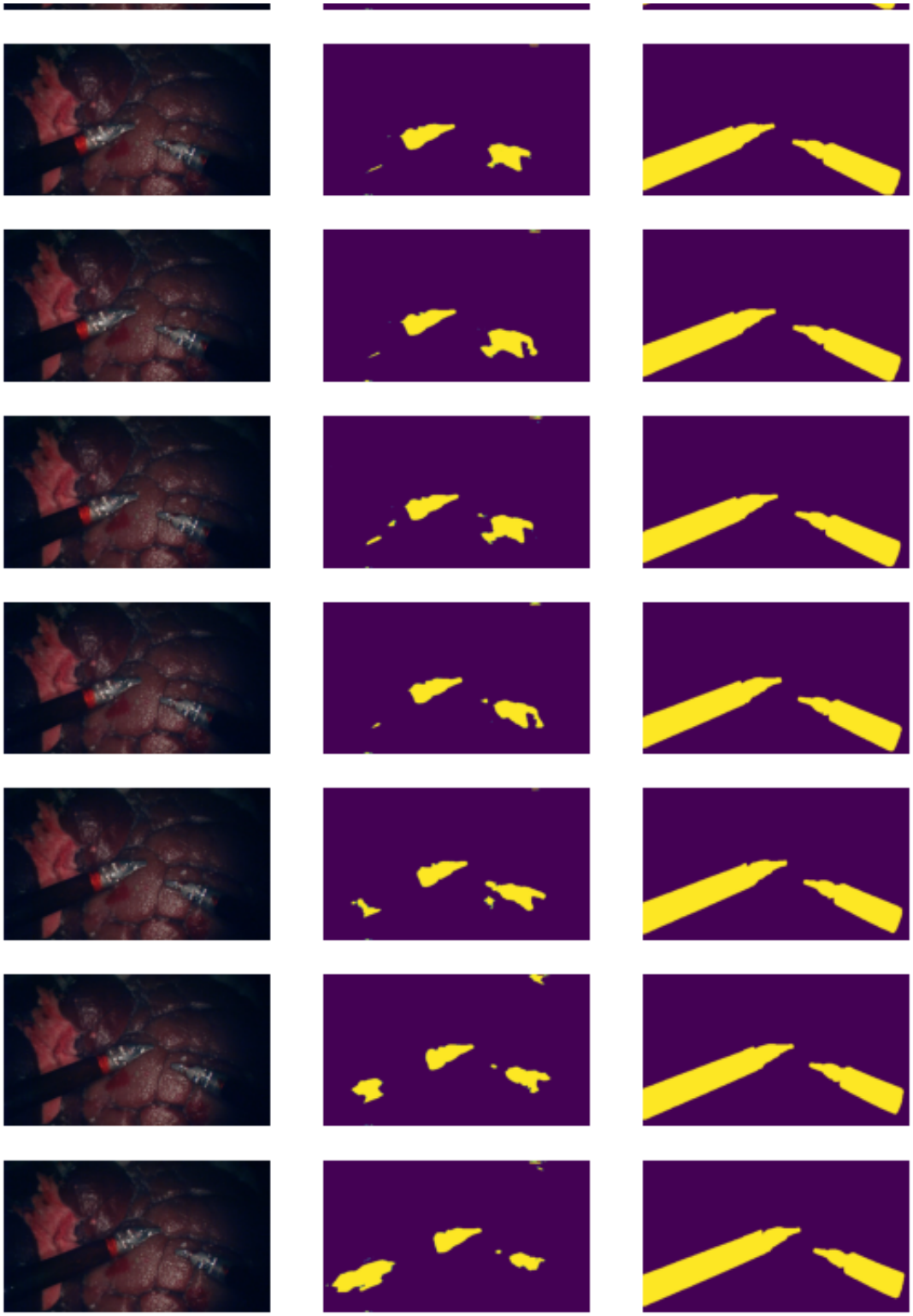




```
segmentation_model_augmented.load_state_dict(torch.load("final_augmen
print(DICE(segmentation_model_augmented, segmentation_test_dataloader_
show_demo(segmentation_model_augmented, segmentation_test_dataloader_l
```

```
0.7269040712714195
```





Problem 2: Transfer Learning

```
## Import VGG and FashionMNIST
from torchvision.models import vgg16
from torchvision.datasets import FashionMNIST

## Specify Batch Size
train_batch_size = 32
test_batch_size = 32

## Specify Image Transforms
img_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

## Download Datasets
train_data = FashionMNIST('./data', transform=img_transform, download=
test_data = FashionMNIST('./data', transform=img_transform, download=T

## Initialize Dataloaders
training_dataloader = DataLoader(train_data, batch_size=train_batch_s
test_dataloader = DataLoader(test_data, batch_size=test_batch_size, sh
```

```
100%|██████████| 26.4M/26.4M [00:02<00:00, 12.4MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 211kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.92MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 13.7MB/s]
```

Model Initialization and Training/Fine-tuning

Complete the rest of the assignment in the notebook below.

```
# ===== Transfer Learning on FashionMNIST with VGG16 =====
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision.models import vgg16
try:
    from torchvision.models import VGG16_Weights
    _IMAGENET_WEIGHTS = VGG16_Weights.IMAGENET1K_V1
except Exception:
    _IMAGENET_WEIGHTS = None  # fallback if older torchvision
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# Fix undefined variable from the given block & (re)build test loader
test_dataset = test_data
test_dataloader = DataLoader(test_dataset, batch_size=test_batch_size

# Utility: evaluate accuracy
def evaluate_accuracy(model, dataloader):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in dataloader:
            # x: (N,1,H,W) -> VGG16 expects 3 channels
            if x.shape[1] == 1:
                x = x.repeat(1, 3, 1, 1)
            x, y = x.to(device), y.to(device)
            logits = model(x)
            preds = logits.argmax(dim=1)
            correct += (preds == y).sum().item()
            total += y.numel()
    return correct / total if total > 0 else 0.0

# Utility: simple training loop
def train_model(model, train_loader, test_loader, criterion, optimize
    model.to(device)
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for x, y in train_loader:
            if x.shape[1] == 1:
                x = x.repeat(1, 3, 1, 1)
            x, y = x.to(device), y.to(device)

            optimizer.zero_grad()
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()

        test_acc = evaluate_accuracy(model, test_loader)
        print(f"{log_prefix}Epoch [{epoch+1}/{epochs}] "
              f"Loss: {running_loss/len(train_loader):.4f} "
              f"| Test Acc: {test_acc*100:.2f}%")
```

```
    return model
```

```
Device: cuda
```

```
# ------------------------ (1) Train VGG16 from scratch -------------
model_scratch = vgg16(weights=None)  # randomly initialized
# Replace classifier head to output 10 classes for FashionMNIST
in_features = model_scratch.classifier[-1].in_features
model_scratch.classifier[-1] = nn.Linear(in_features, 10)

criterion = nn.CrossEntropyLoss()
optimizer_scratch = optim.Adam(model_scratch.parameters(), lr=1e-4, we

print("\n=== Training VGG16 from scratch on FashionMNIST ===")
model_scratch = train_model(
    model_scratch,
    training_dataloader,
    test_dataloader,
    criterion,
    optimizer_scratch,
    epochs=7,
    log_prefix="[Scratch] "
)
acc_scratch = evaluate_accuracy(model_scratch, test_dataloader)
print(f"[Scratch] Final Test Accuracy: {acc_scratch*100:.2f}%")
```

```
=== Training VGG16 from scratch on FashionMNIST ===
[Scratch] Epoch [1/10] Loss: 0.4232 | Test Acc: 90.28%
[Scratch] Epoch [2/10] Loss: 0.2421 | Test Acc: 91.43%
[Scratch] Epoch [3/10] Loss: 0.1947 | Test Acc: 92.05%
[Scratch] Epoch [4/10] Loss: 0.1647 | Test Acc: 92.94%
[Scratch] Epoch [5/10] Loss: 0.1363 | Test Acc: 92.63%
[Scratch] Epoch [6/10] Loss: 0.1105 | Test Acc: 92.58%
[Scratch] Epoch [7/10] Loss: 0.0894 | Test Acc: 93.37%
```

```
# ------------- (2) Pretrained VGG16, freeze all but the last layer ---
criterion = nn.CrossEntropyLoss()
try:
    # Newer PyTorch versions (>=1.13)
    model_ft = vgg16(weights='IMAGENET1K_V1')
except:
    # Older PyTorch versions (<1.13)
    model_ft = vgg16(pretrained=True)
```

```
# Freeze all parameters
for p in model_ft.parameters():
    p.requires_grad = False

# Replace last layer and train only it
in_features = model_ft.classifier[-1].in_features
model_ft.classifier[-1] = nn.Linear(in_features, 10)

# Ensure only new layer is optimized
params_to_update = [p for p in model_ft.parameters() if p.requires_grad
optimizer_ft = optim.Adam(params_to_update, lr=1e-3, weight_decay=1e-4)

print("\n=== Fine-tuning only the last layer of pretrained VGG16 ===")
model_ft = train_model(
    model_ft,
    training_dataloader,
    test_dataloader,
    criterion,
    optimizer_ft,
    epochs=5,
    log_prefix="[FT-last] "
)
acc_ft = evaluate_accuracy(model_ft, test_dataloader)
print(f"[FT-last] Final Test Accuracy: {acc_ft*100:.2f}%")
```

```
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" t
100%|██████████| 528M/528M [00:06<00:00, 80.6MB/s]

=== Fine-tuning only the last layer of pretrained VGG16 ===
[FT-last] Epoch [1/5] Loss: 0.5950 | Test Acc: 83.27%
[FT-last] Epoch [2/5] Loss: 0.5579 | Test Acc: 84.70%
[FT-last] Epoch [3/5] Loss: 0.5572 | Test Acc: 84.43%
[FT-last] Epoch [4/5] Loss: 0.5605 | Test Acc: 84.54%
[FT-last] Epoch [5/5] Loss: 0.5603 | Test Acc: 85.38%
[FT-last] Final Test Accuracy: 85.38%
```